

Laboratório de Engenharia de Sistemas de Computação

Manual NASCERR: Descrição de Software

Fortaleza, 2025

Sumário

1. Introdução 1
2. Visão Geral 1
3. O Protocolo de Comunicação 2
Mapa de Registradores 3
STAT – Registro de status de operação 4
IDX – Registro do Número de Bytes a Serem Transmitidos 5
TXN – Registro de Dados de Telemetria 6
CMD – Registro de Comando de Experimento 6
MOD Register – Registro de Modo de Operação 7
SLEEP – Registro para Entrada no Modo de Economia de Energia 7
WAKEUP – Registro para Saída do Modo de Economia de Energia 8
GPS – Registro de Dados do GPS 8
TIME – Registro de Dados de TIMESTAMP 8
TEMP – Registro de Dados de TEMPERATURA 8
ERROR_EO – Registro de Erros de OPERAÇÃO 8
ERROR_EPS – Registro de Erros de POWER SUPPLY 8
ACK – Registro de ACKNOWLEDGE 9
4. Descrição do Telecomando 10
Formato do Telecomando 10
5. Descrição da Telemetria 11
Formato da Telemetria 11
6. Fluxo de Realização do Experimento 14
7. O Sistema 15
MASTER – Raspberry Pi 4 (Emulador do OBC) 15
SLAVE – NASCERR (STM32 com FreeRTOS) 18
8. Histórico do Documento 20

1. Introdução

Este manual apresenta o software e protocolo de comunicação desenvolvidos para realização de testes e experimentos do projeto NASCERR. A placa está equipada com um Microcontrolador (MCU), uma FPGA e duas memórias SRAM e foi projetada para ser acoplada a um CubeSat 2U, utilizando uma interface CSKB padrão que fornece alimentação elétrica, sinais de controle e telemetria. A interface lógica utiliza um protocolo de comunicação sobre I2C, organizando as informações na forma de banco de registradores lógicos.

O objetivo deste manual é fornecer orientações sobre as características e procedimentos para utilização do software e protocolo de comunicação.

2. Visão Geral

A seguir, na Figura 1 podemos observar o padrão de conexão CSKB da placa do projeto NASCERR.

CubeSat Kit Bus (CSKB)

ESQ-126-39-G-D ou SSQ-126-21-G-D ou TSW-126-07-G-D (parte fêmea voltada para o topo)

H2	2	4	6	8	10	12	14	16	18	20	22	24	26	28	GND 30	GND 32	34	36	38	40	42	44	46	48	/SD_1 50	/SD_2 52
	1	3	5	7	9	11	13	15	17	19	21	23	25	27	GND 29	31	33	35	37	39	41	43	45	47	49	/SD_3 51
H1	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	50	52
	1	3	5	7	9	11	13	15	17	19	I2C1_CLK 21	I2C1_SDA 23	25	27	29	31	33	35	37	39	I2C2_SDA 41	I2C2_CLK 43	45	5V_1 47	5V_2 49	5V_3 51

Figura 1 – Interface demandada pelo NASCERR de acordo com o conector padrão CSKB.

Pino	Rótulo	Definição
H2.29	GND	Terra.
H2.30	GND	Terra.
H2.32	GND	Terra.
H2.50 - 52	/SD_X[2]	Primeiro, segundo[1] e terceiro[1] sinais de SHUTDOWN.
H1.47	5V_1	Primeiro sinal de 5V.
H1.49	5V_2	Segundo sinal de 5V[1].
H1.51	5V_3	Terceiro sinal de 5V[1].
H1.21	I2C1_CLK	Linha de dados do I2C1[1].
H1.23	I2C1_SDA	Linha de clock do I2C[1].
H1.41	I2C2_SDA	Linha de dados do I2C2.
H1.43	I2C2_CLK	Linha de clock do I2C2.

Tabela 1 – Interface demandada pelo NASCERR de acordo com o conector padrão CSKB.

⚠ ATENÇÃO

- [1]Os sinais de 5V_2, 5V_3, o barramento I2C1 e os sinais /SD_2 e /SD_3 são **comumente desconectados**, mas podem ser conectados quando necessário.

- [2]Os sinais de SHUTDOWN, identificados como /SD_X, são usados para realizar o **desligamento total do sistema**. Quando o sinal de entrada em **/SD_X está em nível HIGH**, todo o sistema do NASCERR é desligado.

Todo o protocolo de comunicação é baseado em registros, que são habilitados para leitura ou escrita, os quais podem ser acessados por padrões de comandos pré-definidos de acordo com a próxima unidade. O sistema é implementado utilizando o sistema operacional de tempo real FreeRTOS, o qual executa Tasks de comunicação, experimento e de verificações.

Para a realização do experimento, conta-se com uma Raspberry Pi que atua como *MASTER* no processo de comunicação, ou seja, emulando o OBC. Ela é responsável por implementar o protocolo de comunicação do master e opera uma interface gráfica que facilita o monitoramento e interação rápida com alguns dos registros do sistema, possibilitando enviar comandos para realização de experimentos, consultar o status do sistema ou consultar os resultados dos experimentos realizados.

3. O Protocolo de Comunicação

O NASCERR atua como um *SLAVE* no barramento I2C, seguindo os seguintes parâmetros:

Parâmetro	Valor
Tamanho do Endereço	7-bits
Endereço do Barramento (ADDR)	**0x55**
Frequência de operação (kHz)	100

Tabela 2 – Configurações do SLAVE NASCERR no barramento I2C.

A interface mostrada a seguir utilizada os seguintes pinos do CDKB:

- **H1.43**: **SCL**: sinalização de clock do barramento I2C.
- **H1.41**: **SDA**: sinalização de dados do barramento I2C.

O protocolo implementado utiliza uma abordagem baseada em mapeamento de registradores, onde uma solicitação de escrita no I2C representa um comando e uma solicitação de leitura representa o envio da resposta, conforme mostrado na Figura 2 abaixo:

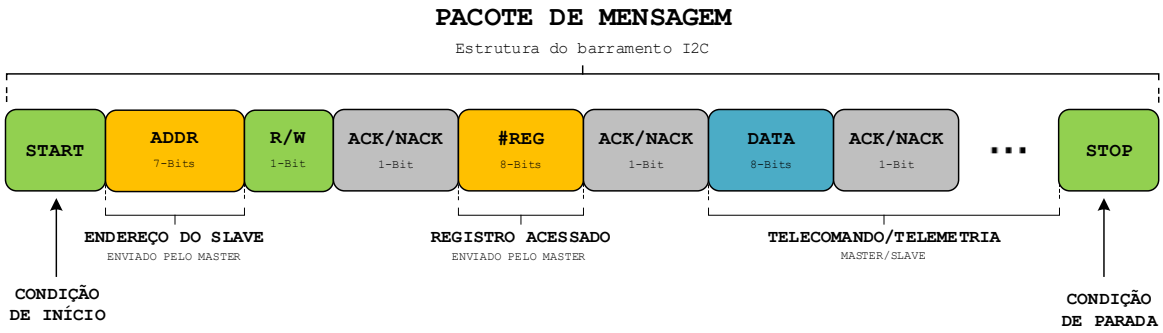


Figura 2 – Representação gráfica do pacote no barramento I2C.

Para garantir a integridade dos dados transmitidos pelo barramento I2C, todos os dados enviados e recebidos utilizam um *checksum* de 16 bits (CRC16). O processo de transmissão consiste na anexação do CRC ao final da mensagem, enquanto o receptor calcula o CRC dos dados recebidos e compara com o valor anexado. Caso os valores não coincidam, ocorre uma falha de transmissão.

- **START:** Condição de start do I2C;
- **ADDR (7 bits): 0x55** – Endereço da carga útil no I2C;
- **R/W(1-bit):** Bit de solicitação de leitura / escrita;
- **#REG(8 bits):** Registro acessado;
- **ACK/NACK(1 bit):** Bit de ACK/NACK no I2C;
- **DATA [1...n]:** Pacote de dados de Telecomando/Telemetria (8 bits);
- **STOP:** Condição de stop do I2C.

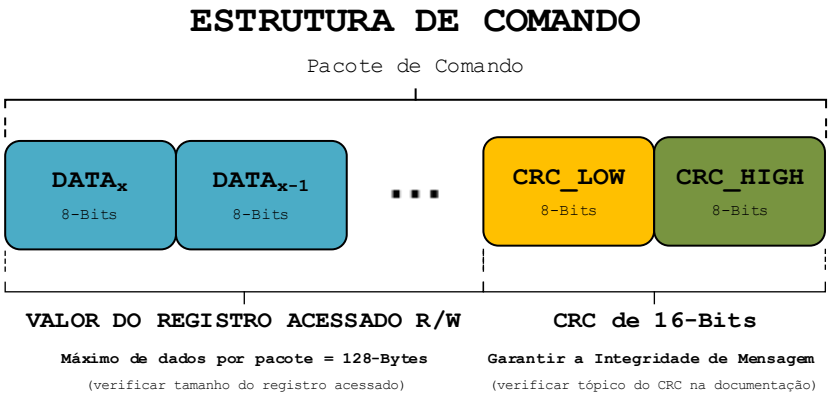


Figura 3 – Representação gráfica da estrutura do pacote com CRC16.

Cada pacote enviado pelo barramento I2C possui a seguinte estrutura:

Dados:

- Representados por **$DATA_x$, $DATA_{(x-1)}$, ...,** com cada segmento contendo 8 bits.
- O número de segmentos pode variar dependendo da mensagem a ser transmitida.

CRC:

- Dividido em duas partes:
  - **CRC_HIGH**: Byte mais significativo do CRC (8 bits).
  - **CRC_LOW**: Byte menos significativo do CRC (8 bits).
- É calculado utilizando o algoritmo padrão CRC-16 com o polinômio **0xA001**.

Mapa de Registradores

O NASCERR possui os registradores presentes na Tabela 3 acessíveis via I2C. Nos tópicos seguintes está contida uma descrição sobre cada um dos registros

Nome	Reg Addr	R/W	Tamanho[3]	Descrição
STAT	0x01	R	4	Registro de status de operação
IDX	0x02	R	4	Registro do número de bytes a serem transmitidos

TXN	0x03	R	154	Registro de dados de TELEMETRIA indicado por IDX
CMD	0x04	W	128	Registro de TELECOMANDO de experimento
MODO	0x05	R	1	Registro de MODO de operação
SLEEP	0x06	W	1	Registro do modo SLEEP
WAKEUP	0x07	W	1	Registro do modo WAKEUP
GPS	0x08	W	TBF[4]	Registro de dados do GPS
TIME	0x09	W	TBF[4]	Registro de dados de TIMESTAMP
TEMP	0x0A	W	TBF[4]	Registro de dados de TEMPERATURA
ERROR_EO	0x0B	R	TBF[4]	Registro de erros de OPERAÇÃO
ERROR_EP	0x0C	R	TBF[4]	Registro de erros de POWER SUPPLY
ACK	0x0D	W	2	Registro de ACKNOWLEDGE
[3]Tamanho referente ao número de bytes				
[4]To Be Featured				

Tabela 3 – Tabela de registradores acessíveis via I2C.

STAT – Registro de status de operação

Os registros de **STAT** estão organizados em duas categorias principais:

- **OP Register (Registro de Operação):** Identifica o status operacional do sistema.
- **Request Register (Registro de Requisições):** Gerencia solicitações realizadas ao sistema.

OP Register – Registro de Operação

O registro de operação indica o estado atual do sistema.

Nome	Status Code	Descrição
OP_OK	0xAA	Registro para status OK (IDLE)
OP_EXPERIMENT	0xAB	Registro para status de EXPERIMENTO EM ANDAMENTO
OP_RECEIVE_COMMAND	0xAC	Registro para status de RECEBIMENTO DE COMANDO
OP_TRANSMIT_RESPONSE	0xAD	Registro para status de TRANSMISSÃO DE RESPOSTA EM ANDAMENTO
OP_TRANSMIT_FINISH	0xAE	Registro para status de ÚLTIMO PACOTE DE TRANSMISSÃO PRONTO
OP_BUSY	0xAF	Registro para status de OCUPADO
OP_FINISH	0xB0	Registro para status de OPERAÇÃO CONCLUÍDA
OP_FAIL	0xB1	Registro para status finalizado com ERROR
OP_ASLEEP	0xB2	Registro para status de SISTEMA DORMINDO

Tabela 4 – Status que o sistema pode assumir.

Uso

- **OP_OK:** Indica que o sistema está pronto para operar, em estado de repouso (IDLE).
- **OP_EXPERIMENT:** Representa que o sistema está executando um experimento em andamento.

- **OP_RECEIVE_COMMAND:** Indica que o sistema recebeu um comando externo.
- **OP_TRANSMIT_RESPONSE:** Reflete que o sistema está enviando pacotes de resposta para uma solicitação.
- **OP_TRANSMIT_FINISH:** Sinaliza que o último pacote de transmissão foi preparado e está pronto.
- **OP_BUSY:** Indica que o sistema está ocupado e não pode aceitar novas requisições no momento.
- **OP_FINISH:** Informa que a operação foi concluída com sucesso.
- **OP_FAIL:** Indica que houve uma falha durante a execução da operação.
- **OP_ASLEEP:** Representa que o sistema desativou os periféricos de modo a entrar no modo de baixo consumo de energia.

Request Register – Registro de Requisições

O registro de requisição especifica as solicitações feitas ao sistema para aquisição de dados ou transmissão de resultados.

Nome	Status Code	Descrição
REQ_NOREQUEST	0xAA	Sem requisição para o MASTER
REQ_TXRESULT	0xAB	Requisição de Transmissão de resultado do experimento
REQ_GPS	0xAC	Requisição dos dados de GPS
REQ_TIME	0xAD	Requisição dos dados de tempo
REQ_TEMP	0xAE	Requisição dos dados de temperatura

Tabela 5 – Status de requisições do sistema.

Uso

- **REQ_NOREQUEST:** Utilizado como valor padrão para indicar que nenhuma solicitação está ativa.
- **REQ_TXRESULT:** Solicita que os resultados do experimento sejam transmitidos.
- **REQ_GPS:** Coleta dados de localização e tempo via GPS.
- **REQ_TIME:** Obtém informações relacionadas ao tempo do sistema.
- **REQ_TEMP:** Coleta dados de sensores de temperatura.

- **NOTA: A FORMA DE CONSULTA E ESTRUTURA DO STAT ESTÁ DETALHADA NO FLUXO DE COMUNICAÇÃO.**

IDX – Registro do Número de Bytes a Serem Transmitidos

O registro IDX indica o número total de bytes de dados de telemetria que precisam ser transmitidos no pacote.

Uso

- **Utilização:**

- Este registro é lido pelo sistema para determinar o tamanho total da transmissão de telemetria. **Tamanho = 4 bytes**.
- Os 4 bytes do registro são enviados na ordem do mais significativo para o menos significativo (big-endian).

▪ **Exemplo:**
- Se o valor armazenado no **IDX** for **0x00001000** (4 bytes), isso indica que o sistema deve transmitir **4096 bytes** de telemetria, no pacote corrente.

TXN – Registro de Dados de Telemetria

O registro onde os dados de telemetria são armazenados para serem transmitidos.

Uso

▪ **Utilização:**
- O sistema lê o registro **TXN** em blocos de dados de acordo com o **Registro de Status de Operação**, já descrito anteriormente, até que o valor especificado em **IDX**, que se refere à quantidade de bytes no pacote, seja alcançado.
- O registro tem um tamanho máximo de **152 bytes** úteis, desconsiderando o CRC, e os dados são transmitidos em blocos consecutivos com esse tamanho.

▪ **Exemplo:**
- Se o **REQUEST REGISTER** estiver indicando a solicitação da transmissão do resultado dos experimentos (*REQ_TXRESULT*) e **OP REGISTER** estiver indicando que um pacote de resposta ou que o último pacote de resposta está pronto para transmissão (*OP_TRANSMIT_RESPONSE* ou *OP_TRANSMIT_FINISH*), então o registro **IDX** será consultado, indicando o tamanho total do pacote a ser lido pelo MASTER.
- Um pacote parcial será usado para os bytes restantes, se o tamanho total de dados a serem transmitidos não for múltiplo de 152.
- O registro de **ACK**, descrito logo mais, auxilia no processo de transmissão por pacotes, confirmado se o MASTER conseguiu ou não realizar a leitura e decodificação do registro de **TXN**.

▪ **NOTA: A ORGANIZAÇÃO E OS DETALHES DOS DADOS DE TELEMETRIA ESTÃO DESCRITOS NO TÓPICO DEDICADO À TELEMETRIA.**

CMD – Registro de Comando de Experimento

Esse registro recebe comandos externos para passar parâmetros e agendar a execução de experimentos do **NASCERR**.

Uso

▪ **Utilização:**

- O sistema escreve no registro CMD e o NASCERR processa esses comandos recebidos e realizar as operações solicitadas.
- Permite que o sistema configure e inicie experimentos pré-definidos, ajustando parâmetros de execução.

▪ **Exemplo:**
- Um comando pode configurar:
  - A quantidade de dados a serem escritos na memória.
  - O tipo de algoritmo de correção de erro a ser utilizado.
  - A quantidade de repetições para as operações de escrita e leitura.

▪ **NOTA: A ESTRUTURA DETALHADA DOS COMANDOS DE TELECOMANDO, INCLUINDO SEUS PARÂMETROS E FUNCIONALIDADES, É APRESENTADA NO TÓPICO DEDICADO AO TELECOMANDO.**

MOD Register – Registro de Modo de Operação

O registro de modo de operação indica qual operação do NASCERR foi configurado, determinando se operações de bypass ou correções de erro estão sendo aplicadas. Este comando é só de **LEITURA**, os modos de operação são configurados pelo registro CMD.

Nome	Mode Code	Descrição
UNDEFINED_MOD	0x00	Modo indefinido
BYPASS_1	0x01	Operação de bypass para SRAM 1
BYPASS_2	0x02	Operação de bypass para SRAM 2
TBEC	0x03	Código de Correção de Erros TBEC_RSC em ambas as SRAMs
MRSC	0x07	Código de Correção de Erros MRSC em ambas as SRAMs

Tabela 6 – Modelos de operação do sistema, tipos de experimentos que podem ser realizados.

Uso

- **UNDEFINED_MOD:** Indica que o NASCERR está em um estado indefinido ou sem configuração inicial.
- **BYPASS_1:** Desativa a correção de erros para operar diretamente na SRAM 1.
- **BYPASS_2:** Desativa a correção de erros para operar diretamente na SRAM 2.
- **TBEC:** Habilita o código de correção de erros TBEC_RSC em ambas as SRAMs.
- **MRSC:** Habilita o código de correção de erros MRSC em ambas as SRAMs para maior robustez.

SLEEP – Registro para Entrada no Modo de Economia de Energia

O registro de *sleep* indica se o sistema deve iniciar o processo de transição para o modo de baixo consumo de energia. Este comando é exclusivamente de **ESCRITA**. Quando esta operação do NASCERR está devidamente configurada, o sistema entra no modo de baixo consumo. Nesse estado, a alimentação da FPGA e das memórias SRAM é interrompida.

OP_STATUS	Activation Code	Descrição
OP_ASLEEP	0xBB	Indica que o sistema deve entrar em operação de baixo consumo de energia.

Tabela 7 - Valor em hexadecimal para ativação do Sleep.

Somente a escrita do valor definido acima ocasionará a transição para o estado de baixo consumo de energia.

WAKEUP – Registro para Saída do Modo de Economia de Energia

O registro de *wakeup* indica se o sistema deve iniciar a transição do modo de baixo consumo de energia para o estado *idle*. Este comando é exclusivamente de **ESCRITA**. Com a operação do NASCERR configurada, o modo de baixo consumo de energia é desativado. Quando isso ocorre, a alimentação da FPGA e das memórias SRAM é restabelecida.

OP_STATUS	Activation Code	Descrição
OP_OK	0xAA	Indica que o sistema deve desativar o modo de baixo consumo de energia e retornar ao estado idle.

Tabela 8 – Valor em hexadecimal para a ativação do Wake Up.

A transição para o estado *idle* ocorrerá somente mediante a escrita do valor definido acima, e apenas se o sistema estiver previamente em modo de baixo consumo de energia.

GPS – Registro de Dados do GPS

TBF (*To Be Featured*)

TIME – Registro de Dados de TIMESTAMP

TBF (*To Be Featured*)

TEMP – Registro de Dados de TEMPERATURA

TBF (*To Be Featured*)

ERROR_EO – Registro de Erros de OPERAÇÃO

TBF (*To Be Featured*)

ERROR_EPS – Registro de Erros de POWER SUPPLY

TBF (*To Be Featured*)

ACK – Registro de ACKNOWLEDGE

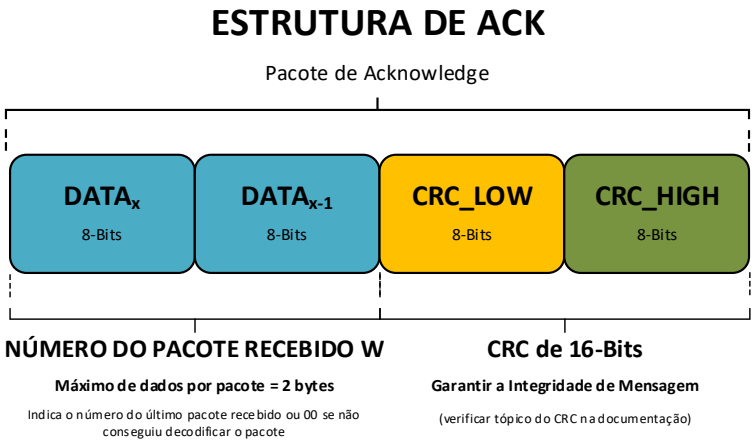


Figura 4 – Representação gráfica da estrutura do pacote com CRC16.

Registro que confirma ao NASCERR que o MASTER conseguiu ler e processar um pacote de **TXN**, mediante a escrita de um valor específico no registro **ACK**.

Uso

- **Função:** Para indicar ao NASCERR que o master recebeu e decodificou corretamente o pacote **TXN** o valor do **NÚMERO DO PACOTE** que acabou de ser lido é escrito no registro **ACK**.
- **Utilização:**
  - Ao escrever qualquer valor diferente do **NÚMERO DO PACOTE** que acabou de ser recebido o SLAVE irá considerar que o MASTER ainda não recebeu o pacote ou recebeu, mas não conseguiu decodificar. Caso o MASTER tenha recebido o pacote, mas não conseguido decodificar escreverá **0x00** no registro **ACK**. Isso implicará em três tentativas de retransmissão do pacote ao MASTER, ou seja, o NASCERR irá escrever o mesmo pacote a ser transmitido no registro **TXN**.
  - Após escrever o pacote **TXN** o SLAVE aguarda o recebimento do **ACK** por um **TIME_OUT** de até 10 segundos.
  - Após atingir o **TIME_OUT** ou **três tentativas** de envio fracassadas o SLAVE para de preparar os próximos pacotes, indicando ao MASTER que há o último pacote a ser lido com a mudança do registro de operação para **OP_TRANSMIT_FINISH**, mesmo que este não seja o último pacote.
- **Exemplo:**
  - No processo de comunicação o sistema deve escrever o **NÚMERO DO PACOTE** recebido no registro **ACK** ou escrever **0x00** caso o pacote seja recebido, mas o MASTER não consiga decodificá-lo.
  - **Valor Inválido:** Qualquer outro valor diferente de **0x00** ou o **NÚMERO DO PACOTE**, implica ao SLAVE aguardar pelo **ACK** até o **TIME_OUT**.

4. Descrição do Telecomando

O **NASCERR** é responsável por processar comandos enviados pelo sistema do OBC. Ele oferece suporte para um padrão de experimento:

1. **Comandos de Escrita na Memória:** Gerenciam a gravação de padrões de dados específicos na memória SRAM.

Formato do Telecomando

O formato do telecomando é estruturado com base nos seguintes campos:

Campo	Descrição	Tipo	Tamanho
bytes_msg	Número total de bytes na mensagem.	U16	2 bytes
cmd_type	Tipo de comando: escrita na memória ou processamento de imagens.	00 – Sem comando 01 – Escrita programada 02 – Experimento contínuo	1 byte
id	Identificador único da mensagem.	U8	1 byte
data_lenght	Comprimento dos dados em blocos (para comandos de escrita na memória).	U32	4 bytes
repeat	Número de repetições para a operação (aplicável aos comandos de memória).	U16	2 bytes
write_type	Tipo de padrão de escrita (zeros, uns, padrões específicos ou dados aleatórios).	00 – Escreve 0x00 01 – Escreve 0x11 02 – Escreve 0x55 03 – Escreve 0xAA 04 – Escreve dados aleatórios	1 byte
data	Tipo de leitura sequencial (WORD_BY_WORD ou BLOCK_BY_BLOCK).	00 – Sequência palavra a palavra 01 – Sequência bloco a bloco	1 byte
mode	Modo de operação do NASCERR.	(Ver registro MOD)	1 byte
delay	Tempo de espera em milissegundos.	U32 (valor em milissegundos)	4 bytes
offset	Offset de escrita ou leitura de memória	U32	4 bytes

Tabela 9 – Estrutura do telecomando (NASCERR_CTRL).

O log apresentado na Figura 5 refere-se a um comando enviado ao **NASCERR**, detalhando a estrutura do comando e os parâmetros associados. No campo *command_bytes*, é possível observar o comando representado em formato hexadecimal, incluindo o CRC calculado no final para verificação de integridade. Esse campo encapsula todas as informações relevantes da mensagem enviada, como tipo de comando, tamanho dos dados, modo de operação e outros parâmetros críticos, assegurando que o NASCERR receba e processe corretamente o comando. Nos capítulos seguintes será apresentado um fluxo de comunicação contendo o telecomando.



Figura 5 – Log de Telecomando.

5. Descrição da Telemetria

Após a realização do experimento o NASCERR irá retornar como reposta os dados de telemetria resultantes do experimento, esses dados ficam armazenados no registro **TDX**, de onde o OBC deve realizar a leitura seguindo o fluxo de telemetria, apresentado no capítulo a seguir.

Formato da Telemetria

O formato da resposta de telemetria tem como base os seguintes campos:

Campo	Descrição	Tipo	Tamanho
id	Identificador único da mensagem.	U8	1 byte
data_lenght	Comprimento dos dados em blocos (para comandos de escrita na memória).	U32	4 bytes
number_errors	Quantidade de palavras contendo erro identificadas.	U32	4 bytes
corrected_data	Quantidade de palavras com erro corrigidas.	U32	4 bytes
percent_error	Percentual médio de erros	F32	4 bytes
cmd_type	Tipo de comando: escrita na memória ou processamento de imagens.	00 – Sem comando 01 – Escrita programada 02 – Experimento Contínuo	1 byte
number_packages	Quantidade de pacotes necessários para transmitir os dados de resposta (telemetria).	U16	2 bytes

package_number	Número do pacote atual que está sendo transmitido.	U16	2 bytes
error_by_package	Quantidade de descrições de erros (ResDescTypeDef) transmitidas por pacote	U16	2 bytes
***result**	Ponteiro para estrutura ResDescTypeDef.	Estrutura ResDescTypeDef	4 bytes

Tabela 10 – Estrutura do resultado de telemetria (NASCERR_RESPONSE).

A estrutura **ResDescTypeDef** representa uma descrição detalhada das palavras em que foram identificados erros, utilizando o seguinte formato:

Campo	Descrição	Tipo	Tamanho
addr	Endereço de memória da palavra que sofreu algum erro.	U32	4 bytes
result	Posição específica de quais bits na palavra sofreram bitflip.	U32	4 bytes

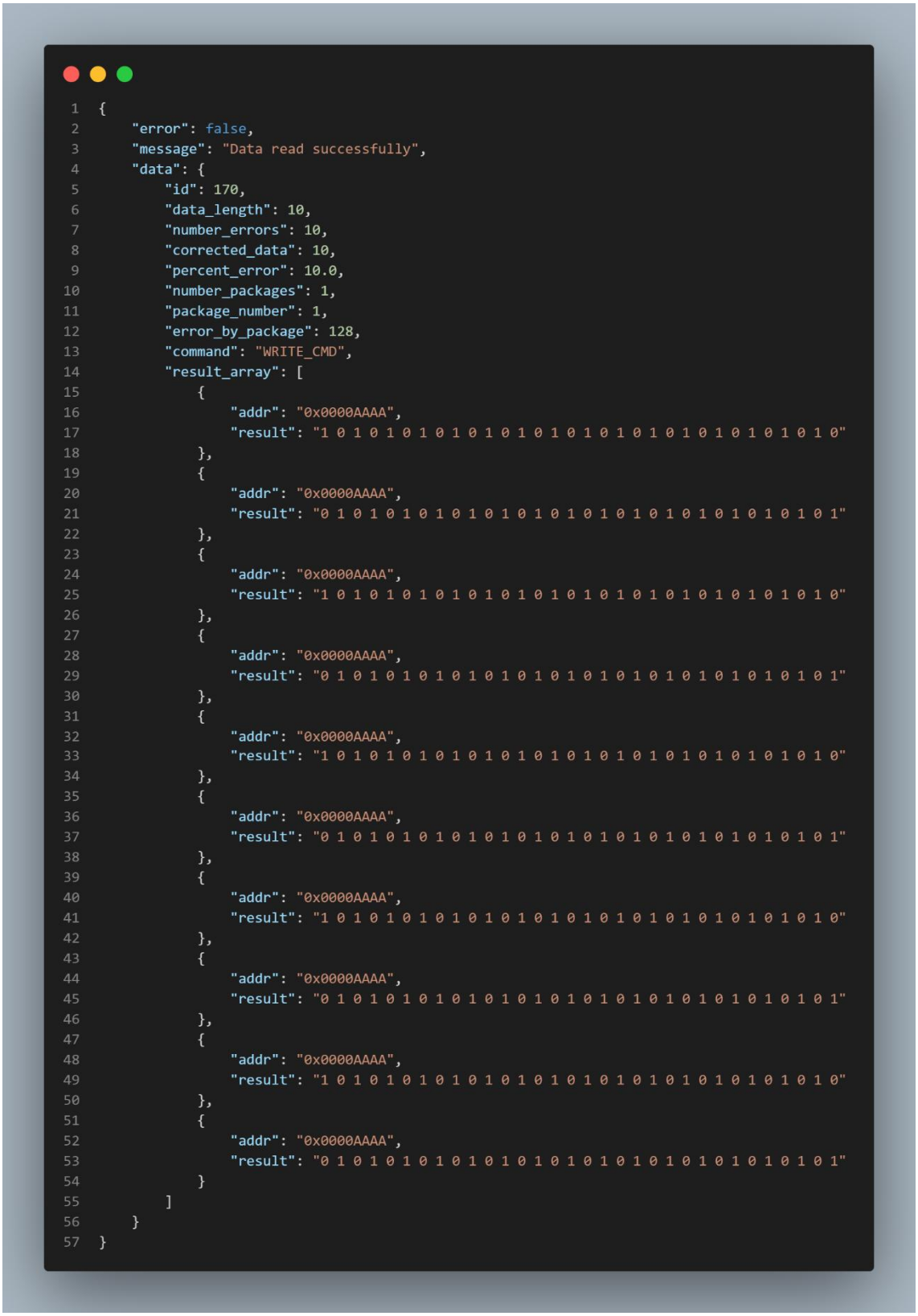
Tabela 11 – Estrutura específica para armazenar informações acerca do experimento.

O log apresentado na Figura 6 refere-se a uma resposta a uma consulta de telemetria ao **NASCERR**, detalhando a estrutura do comando e os parâmetros associados. Esse campo encapsula todas as informações relevantes da mensagem enviada, como tipo de comando, tamanho dos dados, ID do experimento e outros parâmetros críticos, assim, garantido que os resultados do experimento sejam transmitidos de maneira segura e correta. Podemos observar também que a área *result* armazena estruturas **ResDescTypeDef** que detalham os erros, mostrando tanto em que posições da memória ocorreram, como em que bits da palavra esses erros ocorreram. O processo de transmissão dos dados de telemetria por pacotes ocorre de forma que o cabeçalho do payload, descrito na Tabela 10, sempre é enviado no início de cada pacote, alterando-se apenas o *package_number* para o número do pacote corrente e a **ResDescTypeDef**, que representa a estrutura específica de erros, de acordo com a quantidade descrita em **ERROR_BY_PACKAGE.**

- **Exemplo:**
  - Para um dado experimento, caso tenha ocorrido NUM_{ERROR} erros, então, serão transmitidos um total de pacotes definidos pela Equação 1.

$$NUM_{PACKAGE} = \left\lceil \frac{NUM_{ERROR}}{ERROR_BY_PACKAGE} \right\rceil$$

Equação 1 – Número de pacotes.



```
{
    "error": false,
    "message": "Data read successfully",
    "data": {
        "id": 170,
        "data_length": 10,
        "number_errors": 10,
        "corrected_data": 10,
        "percent_error": 10.0,
        "number_packages": 1,
        "package_number": 1,
        "error_by_package": 128,
        "command": "WRITE_CMD",
        "result_array": [
            {
                "addr": "0x0000AAAA",
                "result": "1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0"
            },
            {
                "addr": "0x0000AAAA",
                "result": "0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1"
            },
            {
                "addr": "0x0000AAAA",
                "result": "1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0"
            },
            {
                "addr": "0x0000AAAA",
                "result": "0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1"
            },
            {
                "addr": "0x0000AAAA",
                "result": "1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0"
            },
            {
                "addr": "0x0000AAAA",
                "result": "0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1"
            },
            {
                "addr": "0x0000AAAA",
                "result": "1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0"
            },
            {
                "addr": "0x0000AAAA",
                "result": "0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1"
            },
            {
                "addr": "0x0000AAAA",
                "result": "1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0"
            },
            {
                "addr": "0x0000AAAA",
                "result": "0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1"
            }
        ]
    }
}
```


Figura 6 – Log de resposta de telemetria.

6. Fluxo de Realização do Experimento

A seguir, na Figura 7, é apresentado o fluxo de envio de um telecomando do master para o **NASCERR**, seguido pela recepção dos dados após o processamento e execução do experimento, com o retorno da telemetria sendo transmitido em blocos de dados.

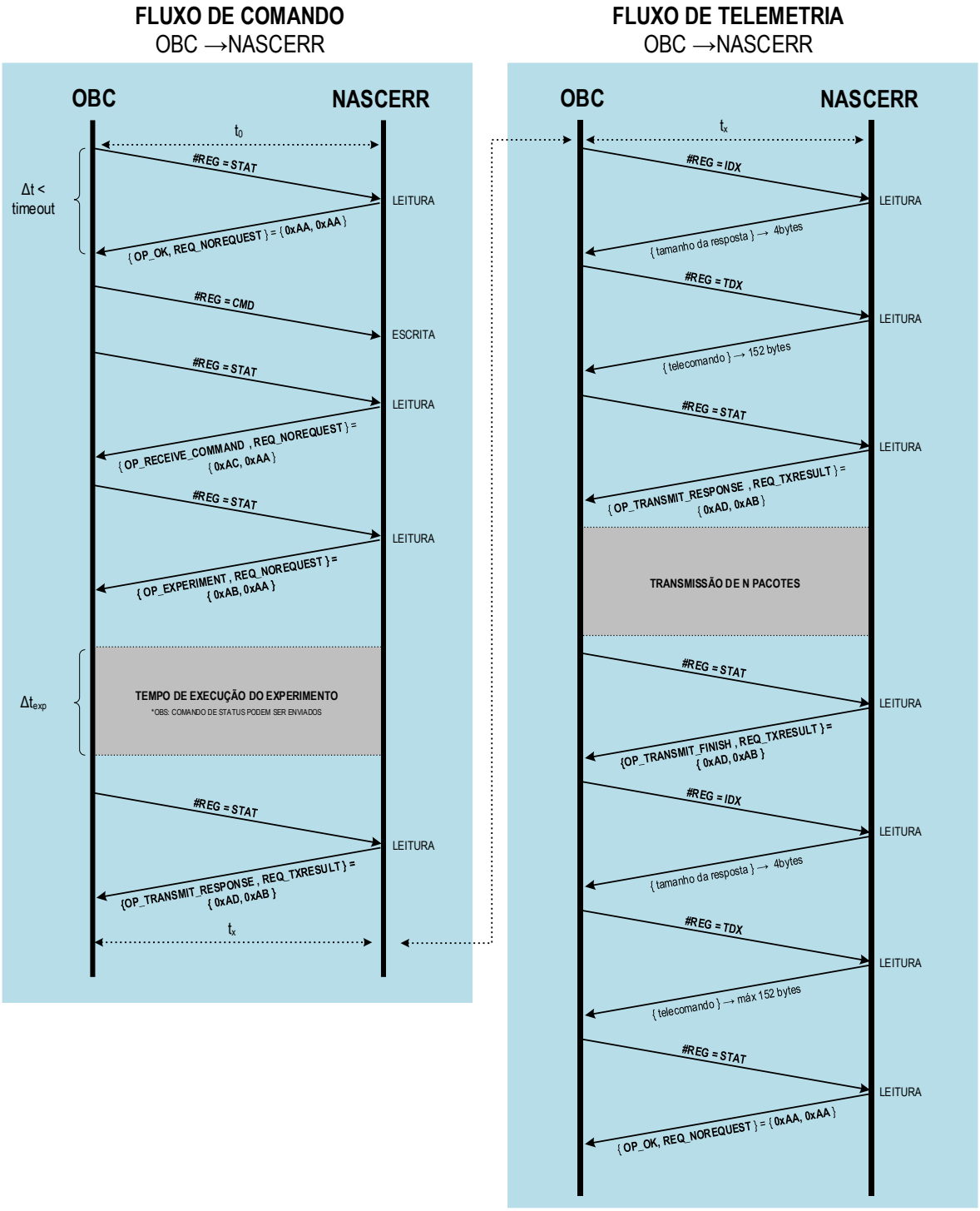


Figura 7 – Fluxo entre comunicação para a realização e obtenção dos resultados do experimento.

7. O Sistema

Como já comentando anteriormente, o sistema é composto por dois componentes, o *MASTER*, implementando em uma Raspberry Pi 4 que é responsável pelo envio de telecomandos e processamento dos dados resultantes do experimento, e o *SLAVE*, implementando na placa do NASCERR e que é responsável pela execução do experimento, tratamento dos dados de telemetria e envio desses dados ao master.

MASTER – Raspberry Pi 4 (Emulador do OBC)

O papel do OBC, *MASTER* da comunicação, simulado em um Raspberry Pi 4, responsável pelo controle dos experimentos e comunicação com o *SLAVE*. Ele executa o sistema operacional Linux e possui interfaces para comunicação I2C, além da implementação do protocolo descrito nos capítulos anteriores para o envio de telecomandos e recepção de dados de telemetria. As principais funções do *MASTER* incluem:

- Enviar Telecomandos: O usuário na interface do Raspberry Pi 4 consegue passar comandos estruturados conforme o protocolo definido e os envia ao STM32 via interface I2C, utilizando os GPIOs 2 e 3, de a cordo com a Figura 8.
- Gerenciamento de Fluxo: Controla o envio dos comandos e aguarda as respostas, garantindo a correta execução dos experimentos, também é possível a realização de experimentos sucessivos na interface gráfica, no modo CONTINUOUS_CMD é possível definir pela interface a quantidade de experimentos a serem realizados, o tempo de cada experimento e outros parâmetros opcionais do comando, todos os comandos enviados também são salvos em um *json*.
- Processamento dos Dados de Telemetria: Recebe as respostas do SLAVE via pacotes, processa as informações de erro, correção e desempenho e expões esses resultados ao usuário via terminal, interface gráfica ou exportando em um arquivo de CSV, todos os resultados de telecomando recebidos também são salvos em um arquivo *json*.
- Interface com o Usuário: Pode fornecer logs, informações acerca do status do SLAVE e um sistema WEB local com interface gráfica implementado para facilitar a interação com o usuário, podendo ser observada nas figuras seguintes.

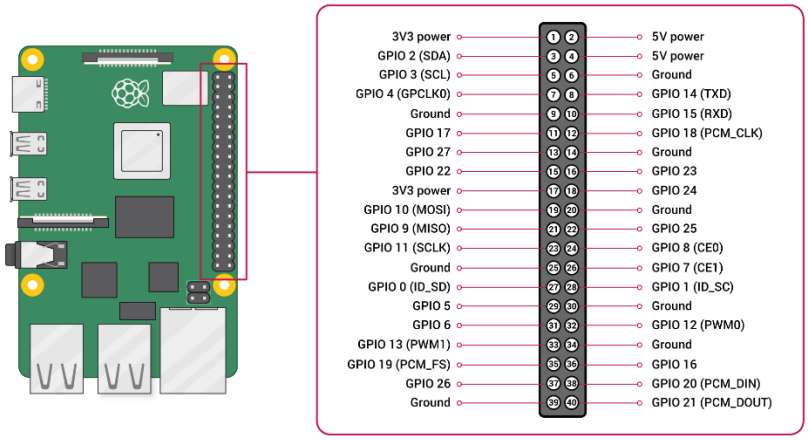


Figura 8 – Pinout da raspberry pi 4.

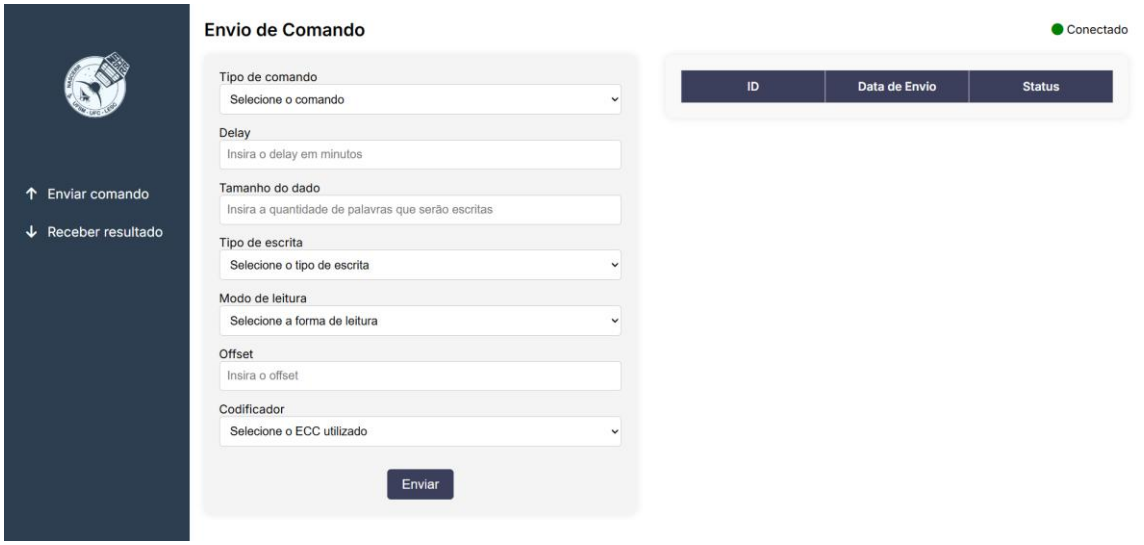


Figura 9 – Interface gráfica do MASTER em estado IDLE.

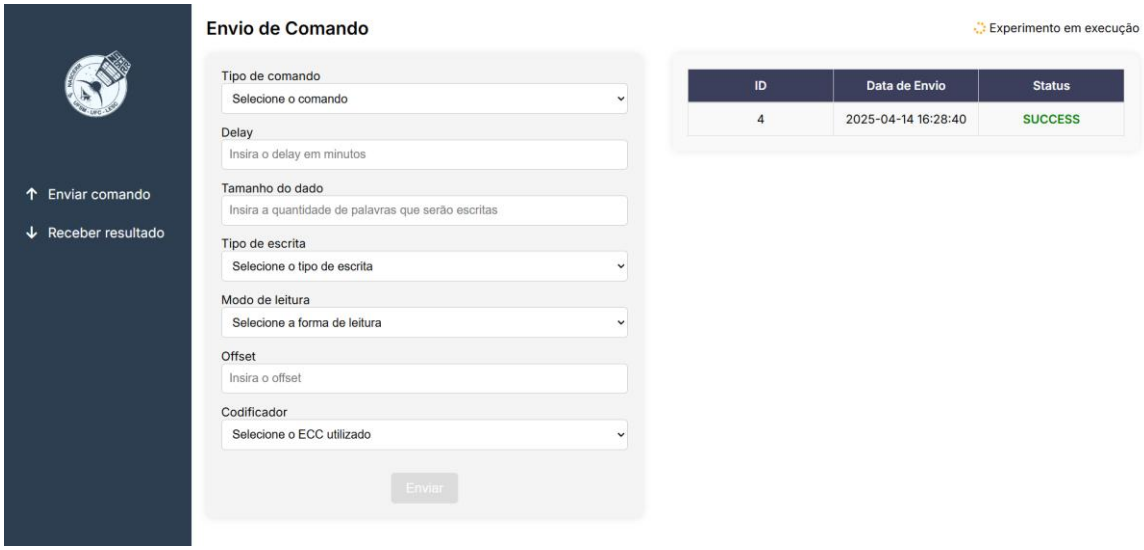


Figura 10 – Interface gráfica do MASTER no momento de realização de algum experimento.

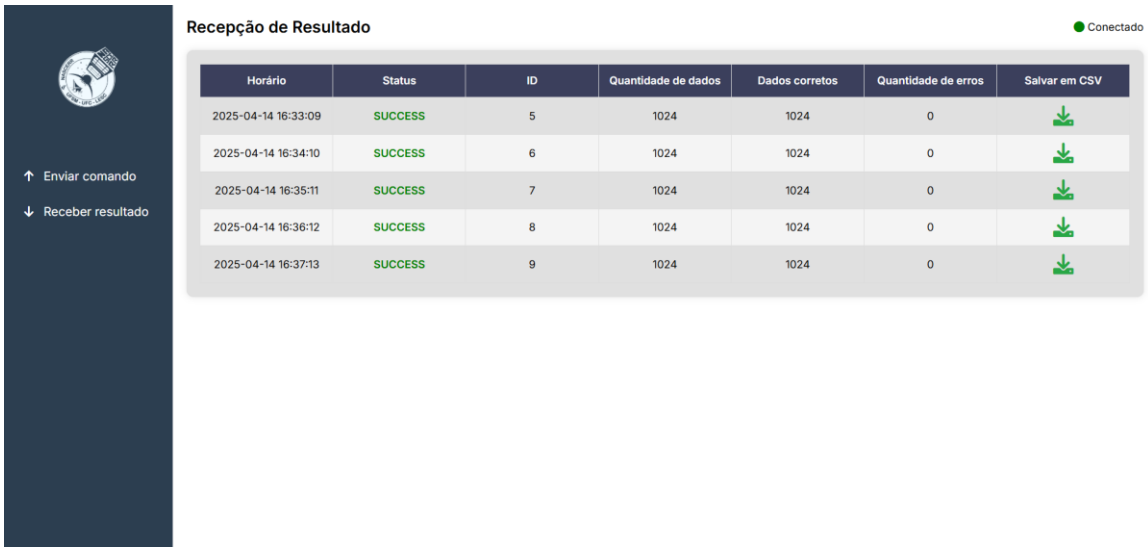


Figura 11 – Interface gráfica do MASTER na tela de recepção de resultados.

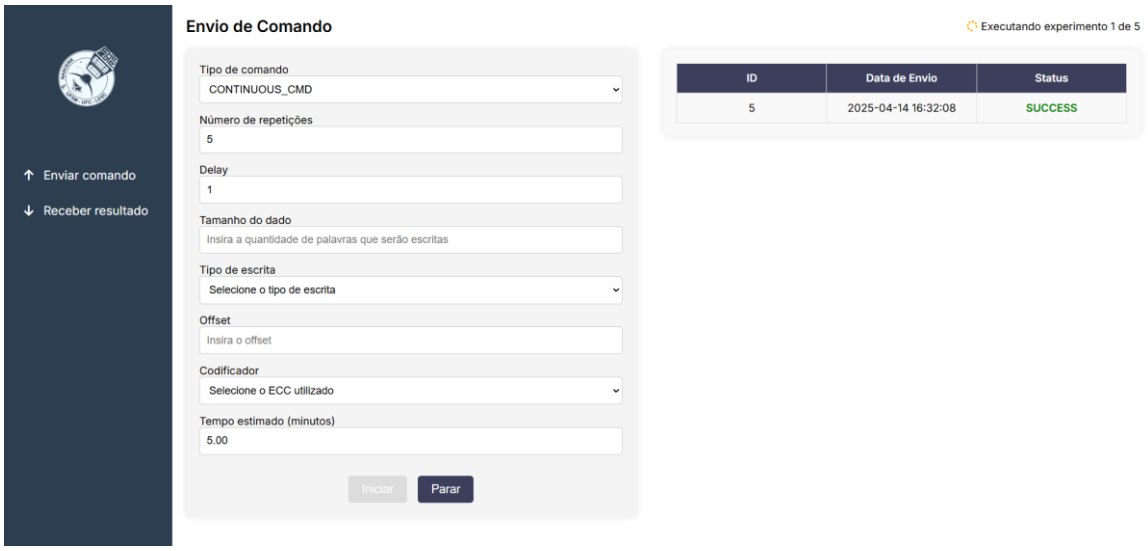


Figura 12 – Interface em modo de execução de experimento contínuo.

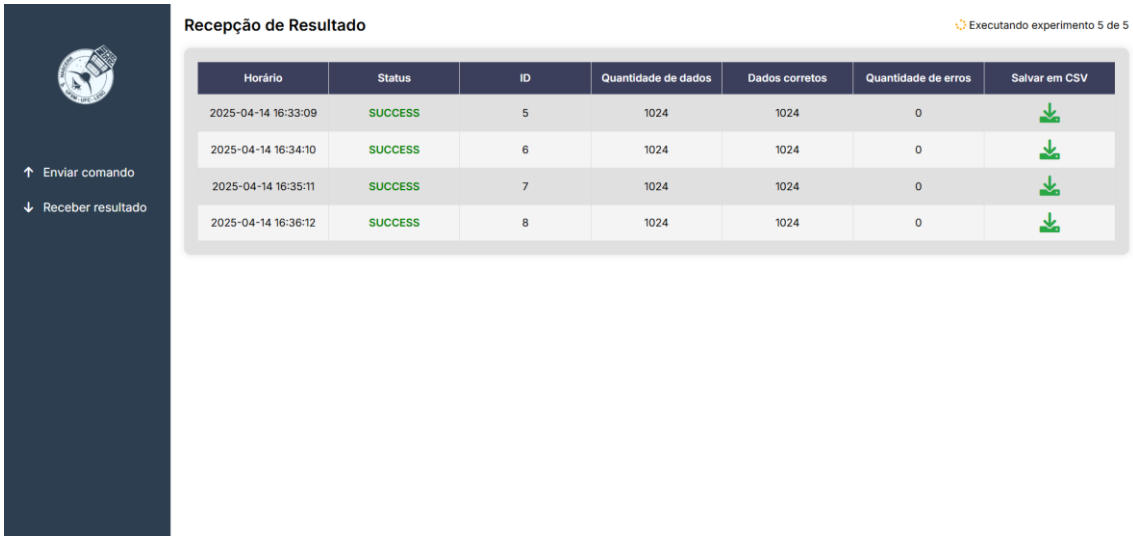


Figura 13 – Interface gráfica de recepção no modo de comando contínuo.

Para inicializar a interface gráfica no Raspberry Pi, basta acessar o ambiente NASCERR_Application, garantir que o Python esteja instalado e proceder com a instalação das bibliotecas listadas no arquivo requirements.txt. Recomenda-se o uso de um ambiente virtual (.venv) para isolar as dependências do sistema. Com o ambiente devidamente configurado, a execução do sistema é feita rodando o arquivo *main.py*. A interface gráfica é então iniciada por meio do **FastAPI**, ficando acessível em rede local por meio do endereço **http://IP_DA_RASPBERRY:8000**.

Também é possível utilizar a Raspberry Pi sem a interface gráfica. Nesse caso, o arquivo OBDH_Emu.py permite o envio de comandos, a consulta do status do sistema e a verificação das respostas dos experimentos diretamente via código. Para isso, são utilizados os métodos da classe **Operation**, que deve ser inicializada com o nome do dispositivo *SLAVE* e seu respectivo endereço no barramento I2C. Após a inicialização, os seguintes métodos podem ser utilizados:

- `send_command()` – Envia um comando ao *SLAVE*, conforme as estruturas definidas anteriormente.
- `get_status()` – Consulta o estado atual do sistema.
- `read_result_byPackages()` – Obtém os resultados do experimento, seguindo o formato estruturado de telemetria.

SLAVE – NASCERR (STM32 com FreeRTOS)

O *SLAVE* é o sistema embarcado NASCERR, baseado em um microcontrolador *STM32* executando *FreeRTOS*. Ele é responsável pela execução dos experimentos e pelo envio dos dados de telemetria ao *MASTER*. Suas principais funções incluem:

- Recebimento e Processamento de Telecomandos: Interpreta os comandos recebidos do Master, identificando o tipo de operação e executando a realização dos experimentos, realiza a escrita de padrões de dados na memória SRAM ou aplica processamento e compressão em imagens pré-carregadas.
- Geração e envio de Telemetria: Após a execução do experimento, coleta dados sobre os erros de memória que ocorreram, correções realizadas e estatísticas relevantes. transmite os dados de telemetria ao Master em pacotes estruturados, seguindo o fluxo de comunicação já apresentado.
- Gerenciamento de Tarefas com FreeRTOS: Utiliza o sistema operacional de tempo real para garantir a execução eficiente e ordenada das tarefas do sistema, no qual gerencia as tarefas e as executa nos momentos propícios, de acordo com o diagrama da Figura 14.

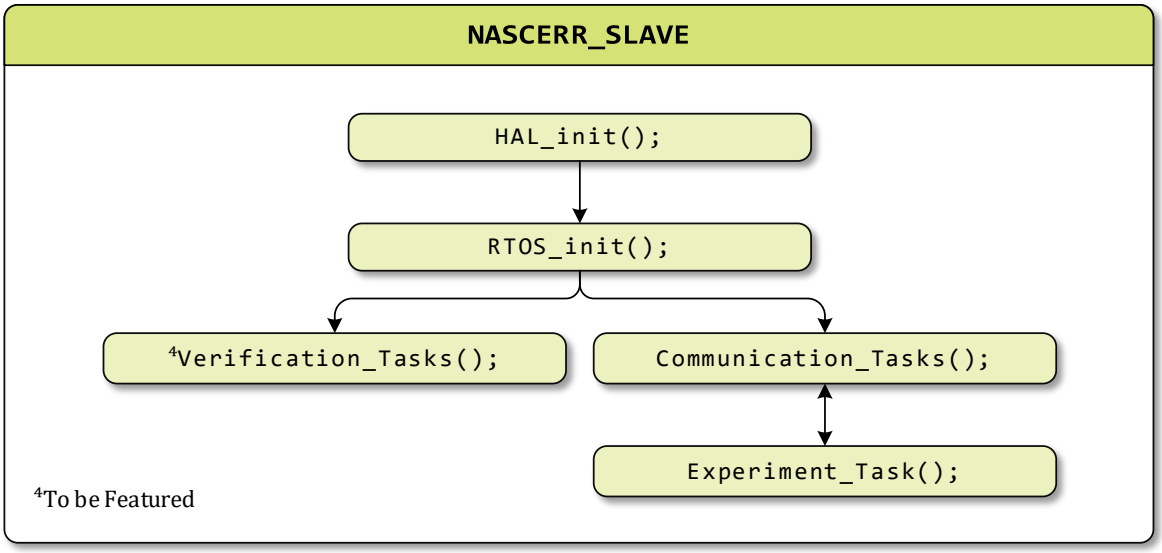


Figura 14 – Diagrama de TASKs do FreeRTOS no STM32.

O processo de inicialização do firmware do *SLAVE* começa com a configuração de todos os periféricos utilizados pelo NASCERR. Em seguida, é inicializado o protocolo de comunicação, por meio da função `i2c_slave_init()`. Após essa etapa, são criadas as tarefas responsáveis pelo tratamento de comandos (`receive_dataResp()`). Por fim, o sistema operacional é inicializado, garantindo o funcionamento das tarefas e a comunicação com o *MASTER*. Após a inicialização, o sistema entra em estado de espera, aguardando a recepção de novos comandos. A verificação da chegada de comandos é realizada pela função `nascerr_protocol_receivedNewCommand()`.

Quando um novo comando é identificado, ele é processado pela função `nascerr_control_castingCMD(NASCERR_CTRL *ctrl, uint8_t *cmd_value)`, que valida e converte os dados conforme a estrutura definida na Tabela 9**.** Após essa etapa, a task `nascerr_experiment()` é iniciada para executar o experimento correspondente. Ao final da execução, a task `send_dataResp()` é ativada, responsável pelo envio dos resultados de telemetria ao *MASTER*.

Esse envio é realizado por meio da função `nascerr_protocol_requestTxResult_byPackages(NASCERR_RESPONSE response, uint16_t error_by_package, uint32_t max_timeout)`, garantindo a transmissão estruturada dos dados coletados.

O arquivo **NASCERR_System.h** permite a configuração do projeto para diferentes cenários de uso, oferecendo maior flexibilidade no desenvolvimento e depuração do sistema. As opções disponíveis são:

- **MOCK_EXPERIMENT**: Simula a execução do experimento, gerando dados fictícios como resultado, sem realizar a operação real de escrita e leitura da memória.
- **MOCK_COMMAND**: Dispensa a comunicação externa para recepção de comandos, montando o comando diretamente no *SLAVE*.
- **ENABLE_LOGS**: Habilita a geração de logs do sistema via interface serial, utilizando uma taxa de transmissão de **6.000.000 Bits/s**.
- **SOFTWARE_DEBUG**: Desativa o **IWDG** (Independent Watchdog Timer), permitindo a depuração do firmware sem que o sistema seja reiniciado.

- **ERROR_BY_PACKAGE**: Define a quantidade de descrições de erros (definidas conforme Tabela 11) que serão transmitidas por pacote de telemetria.

8. Histórico do Documento

Data	Revisão	Alterações	Autor
01/02/25	1.0	Documentação parcial	João Manoel e Marcus Anderson
27/03/25	1.1	Revisão da documentação, correções de erros e melhor descrição.	João Manoel e Cícero Oliveira
14/04/25	1.2	Atualização de interface gráfica	João Manoel e Cícero Oliveira
15/04/25	1.3	Descrição da implementação dos features de baixo consumo e correção de erros.	João Manoel e Cícero Oliveira